ゲームを作る: 1
モグラたたき

プログラミング演習I

2018/06/01

ゲームを作る

ゲームをつくる手順

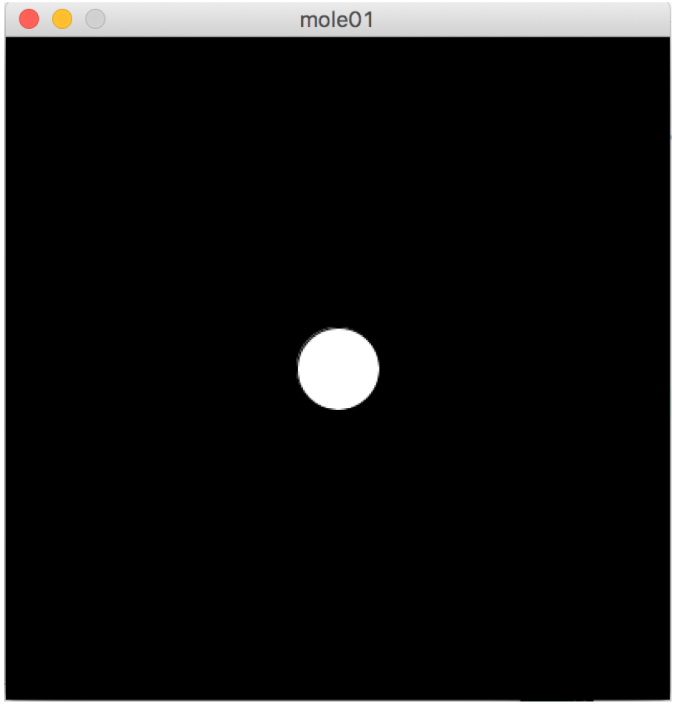
- 作るゲームを考える
  - ゲームのルールを決める（勝ち負け）
  - ゲームの遊び方を決める（操作方法）
  - ゲームのデザインを決める（見た目）
- ゲームを作る
  - 作り方を決める（何で作る、どうやって作る）
  - 作ってみる
  - 遊んでみる、改良する

「モグラたたき」を作る

「モグラ叩き」

- Step1: 表示、操作の原型
- Step2: 当たり判定、スコア
- Step3: モグラの動作
- Step4: 叩くと消える
- Step5: 終了条件
- Step6: シーン切替、勝敗判定
- Step7: シーン追加

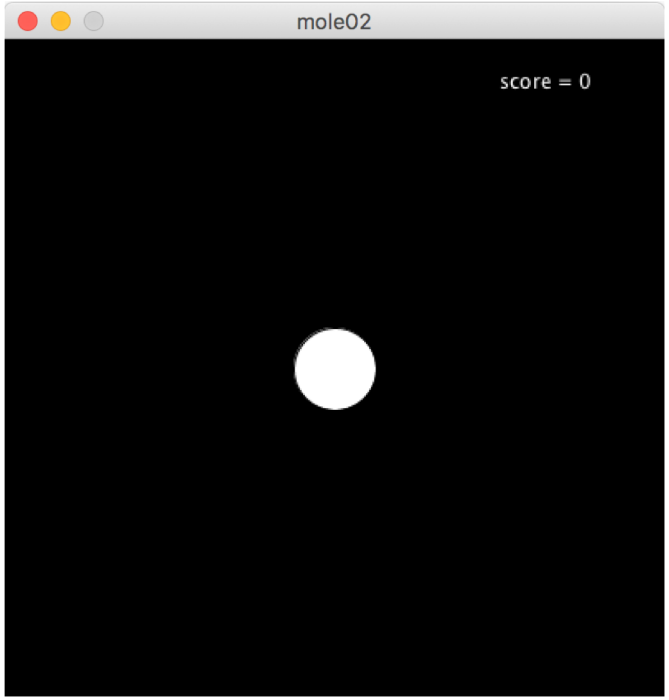
Step1: 表示、操作の原型



ソースコード

```
void setup() {
  size(400, 400);
}

void draw() {
  background(0);

  ellipse(200, 200, 50, 50);

  if (mousePressed) {
    println("Hit!");
  }
}
```

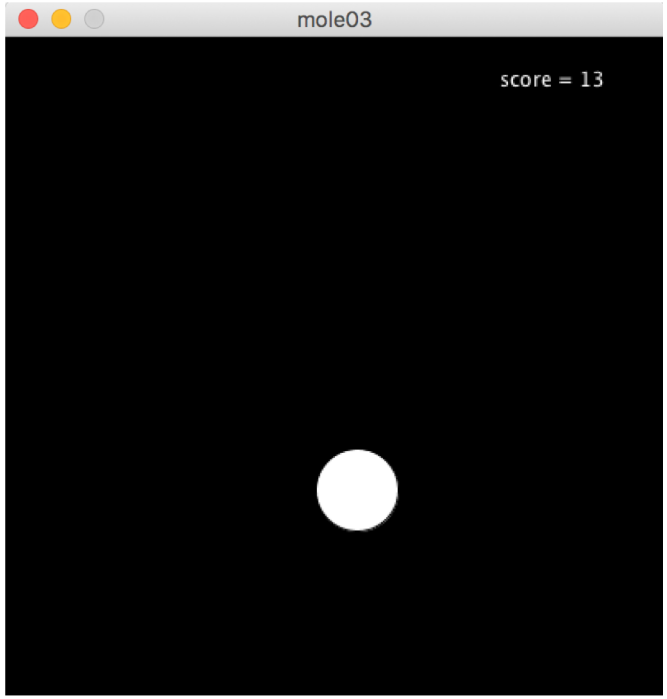
Step2: 当たり判定、スコア



ソースコード

```
float mx, my;
int score;

void setup() {
  size(400, 400);
}

void draw() {
  mx = 200;
  my = 200;

  background(0);
  text("score = " + score, 300, 30);

  ellipse(mx, my, 50, 50);

  if (mousePressed) {
    if (dist(mouseX, mouseY, mx, my) < 50) {
      score++;
    }
  }
}
```

Step3: モグラの動作



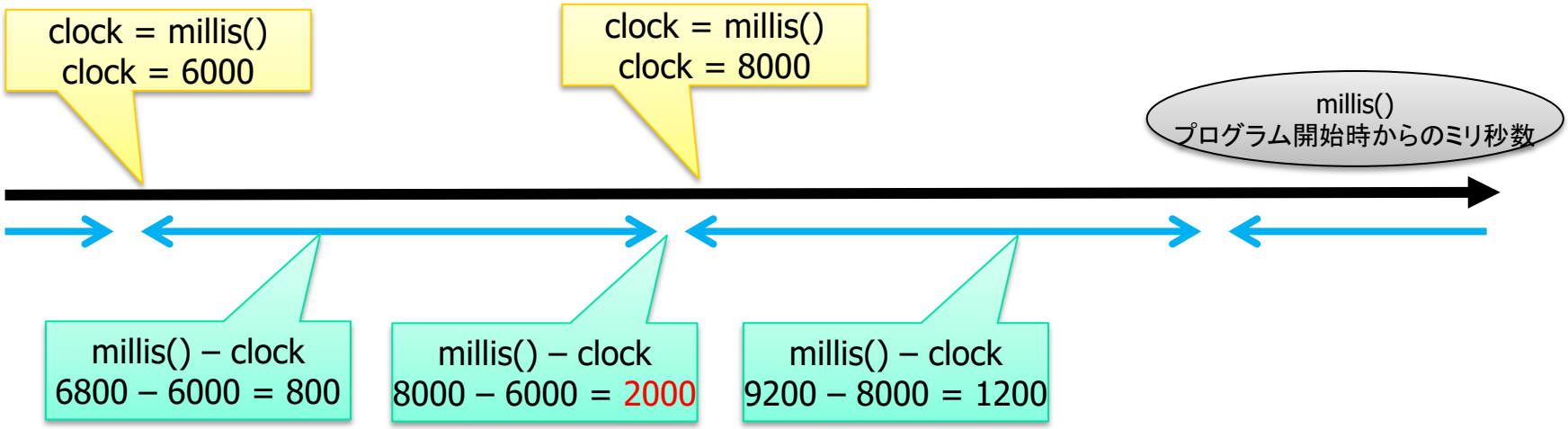
ソースコード

```
float mx, my;
int score;
int clock;

void setup() {
  size(400, 400);
  clock = millis();
}

void draw() {
  if (millis() - clock >= 2000) {
    clock = millis();
    mx = random(0, 400);
    my = random(0, 400);
  }

  background(0);
  text("score = " + score, 300, 30);

  ellipse(mx, my, 50, 50);

  if (mousePressed) {
    if (dist(mouseX, mouseY, mx, my) < 50) {
      score++;
    }
  }
}
```

（コラム）時間間隔の取り方

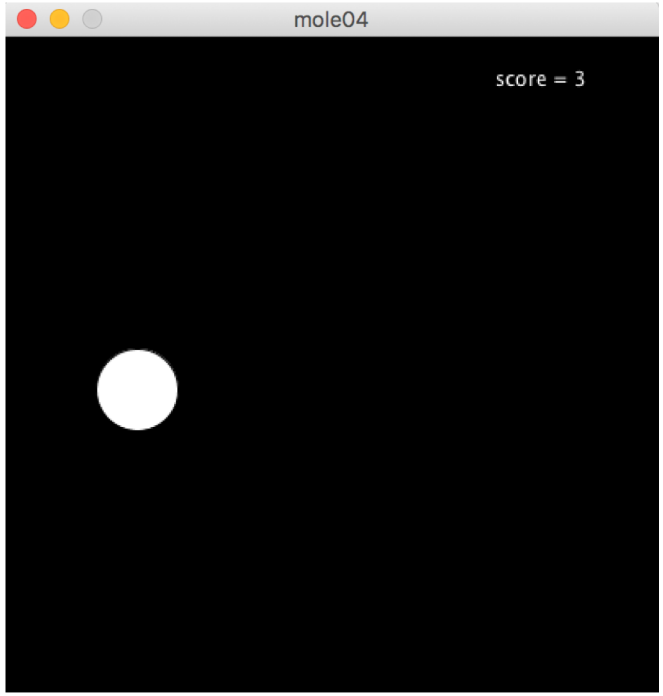
- インターバルを調べる

clock = millis() で前の実行時間を記録



millis() – clock で前の実行時間からの間隔を調べる

Step4: 叩くと消える

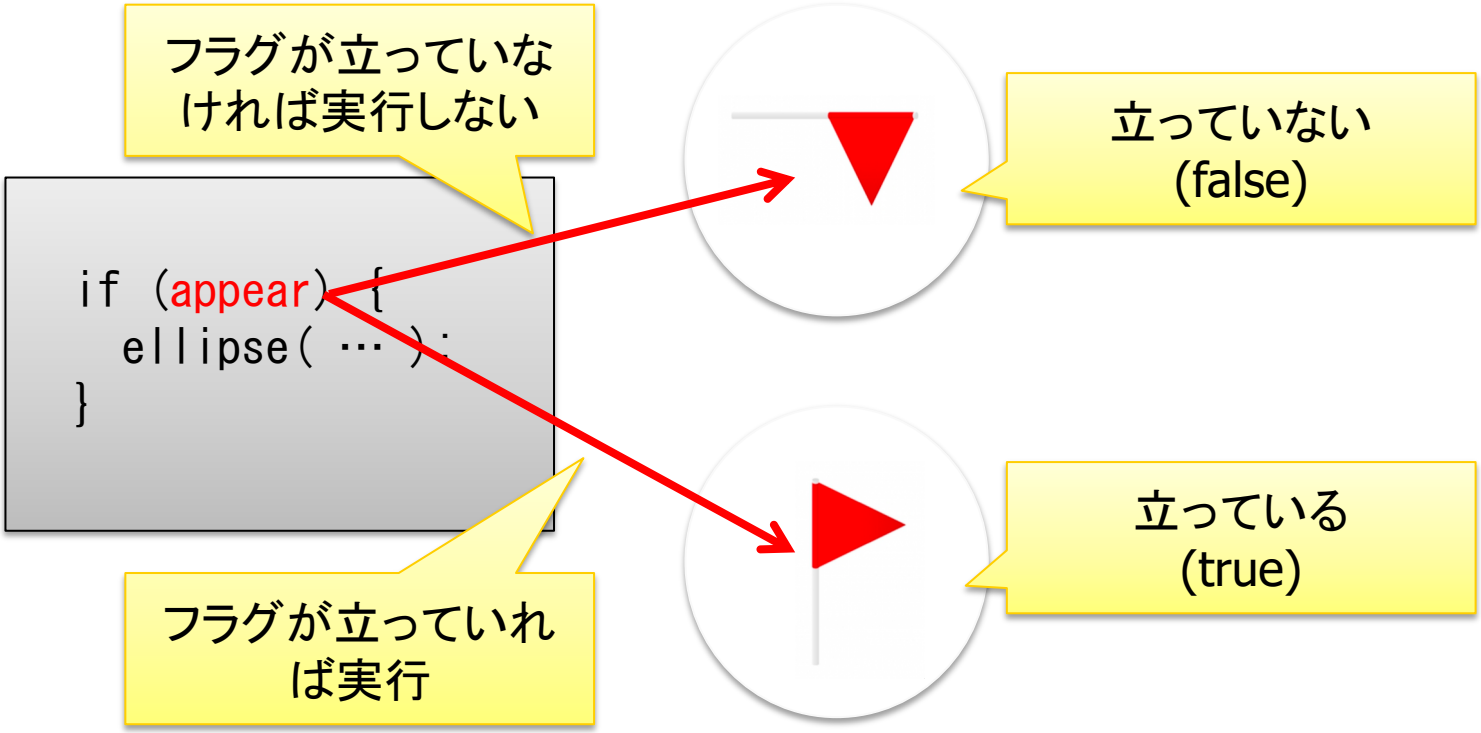


ソースコード

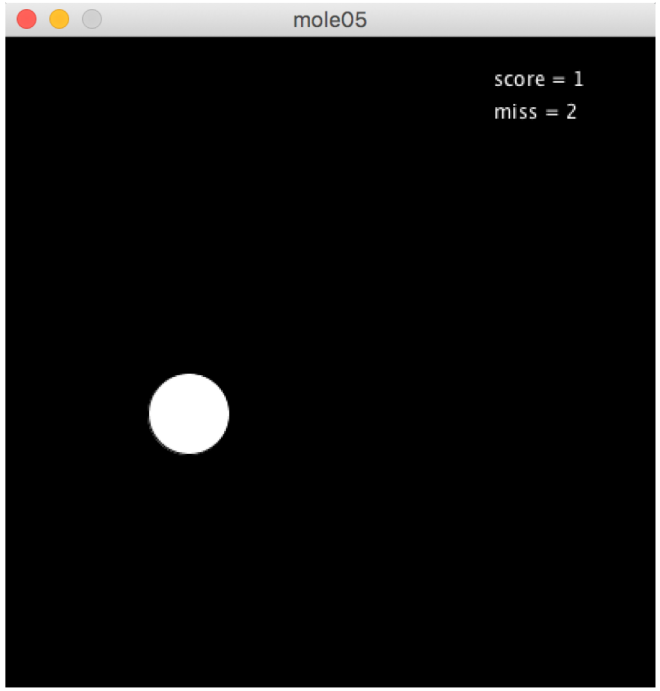
```
int score;
int clock;
boolean appear;

void setup() {
  size(400, 400);
  clock = millis();
}

void draw() {
  if (millis() - clock >= 2000) {
    clock = millis();
    mx = random(0, 400);
    my = random(0, 400);
    appear = true;
  }

  background(0);
  text("score = " + score, 300, 30);

  if (appear) {
    ellipse(mx, my, 50, 50);

    if (mousePressed) {
      if (dist(mouseX, mouseY, mx, my) < 50) {
        score++;
        appear = false;
      }
    }
  }
}
```

（コラム）フラグを使う

- フラグ状態を調べて実行切替



Step5: 終了条件



ソースコード

```
float mx, my;
int score;
int miss;
int clock;
boolean appear;

void setup() {
  size(400, 400);
  clock = millis();
}

void draw() {
  if (score >= 3 || miss >= 3) {
    exit();
  }

  if (millis() - clock >= 2000) {
    clock = millis();
    mx = random(0, 400);
    my = random(0, 400);
    if (appear) {
      miss++;
    }
    else {
      appear = true;
    }
  }

  background(0);
  text("score = " + score, 300, 30);
  text("miss = " + miss, 300, 50);

  if (appear) {
    ellipse(mx, my, 50, 50);

    if (mousePressed) {
      if (dist(mouseX, mouseY, mx, my) < 50) {
        score++;
        appear = false;
      }
    }
  }
}
```

Step6:シーン切替、勝敗判定

メインタブ

```
int scene;
float mx, my;
int score;
int miss;
int clock;
boolean appear;

void setup() {
  size(400, 400);
  clock = millis();
}

void draw() {

  switch (scene) {
  case 0:
    playing();
    break;
  case 1:
    result();
    break;
  }
}
```

playing タブ

```
void playing() {
  if (score >= 3 || miss >= 3) {
    scene++;
  }

  if (millis() - clock >= 2000) {
    clock = millis();
    mx = random(0, 400);
    my = random(0, 400);
    if (appear) {
      miss++;
    } else {
      appear = true;
    }
  }

  background(0);
  text("score = " + score, 300, 30);
  text("miss = " + miss, 300, 50);

  if (appear) {
    ellipse(mx, my, 50, 50);

    if (mousePressed) {
      if (dist(mouseX, mouseY, mx, my) < 50) {
        score++;
        appear = false;
      }
    }
  }
}
```

result タブ

```
void result() {
  background(0);

  if (score == 3) {
    text("You Win!", 200, 200);
  }
  else {
    text("Game Over ... ", 200, 200);
  }
}
```

（コラム）switch文

- 「多分岐」の処理

```
switch(値) {
case 0:
  値が0だった場合の処理
  break;
case 1:
  値が1だった場合の処理
  break;
case 2:
  値が2だった場合の処理
  break;
case 3:
  値が3だった場合の処理
  break;
}
```

if文は2分岐

```
if(条件) {
  条件がtrueの場合の処理
}
else {
  条件がfalseの場合の処理
}
```

Step7:シーン追加

メインタブ

```
int scene;
float mx, my;
int score;
int miss;
int clock;
boolean appear;

void setup() {
  size(400, 400);
  clock = millis();
}

void draw() {
  switch (scene) {
  case 0:
    opening();
    break;
  case 1:
    playing();
    break;
  case 2:
    result();
    break;
  }
}
```

playing タブ

```
void playing() {
  if (score >= 3 || miss >= 3) {
    scene++;
  }

  if (millis() - clock >= 2000) {
    clock = millis();
    mx = random(0, 400);
    my = random(0, 400);
    if (appear) {
      miss++;
    } else {
      appear = true;
    }
  }

  background(0);
  text("score = " + score, 300, 30);
  text("miss = " + miss, 300, 50);

  if (appear) {
    ellipse(mx, my, 50, 50);

    if (mousePressed) {
      if (dist(mouseX, mouseY, mx, my) < 50) {
        score++;
        appear = false;
      }
    }
  }
}
```

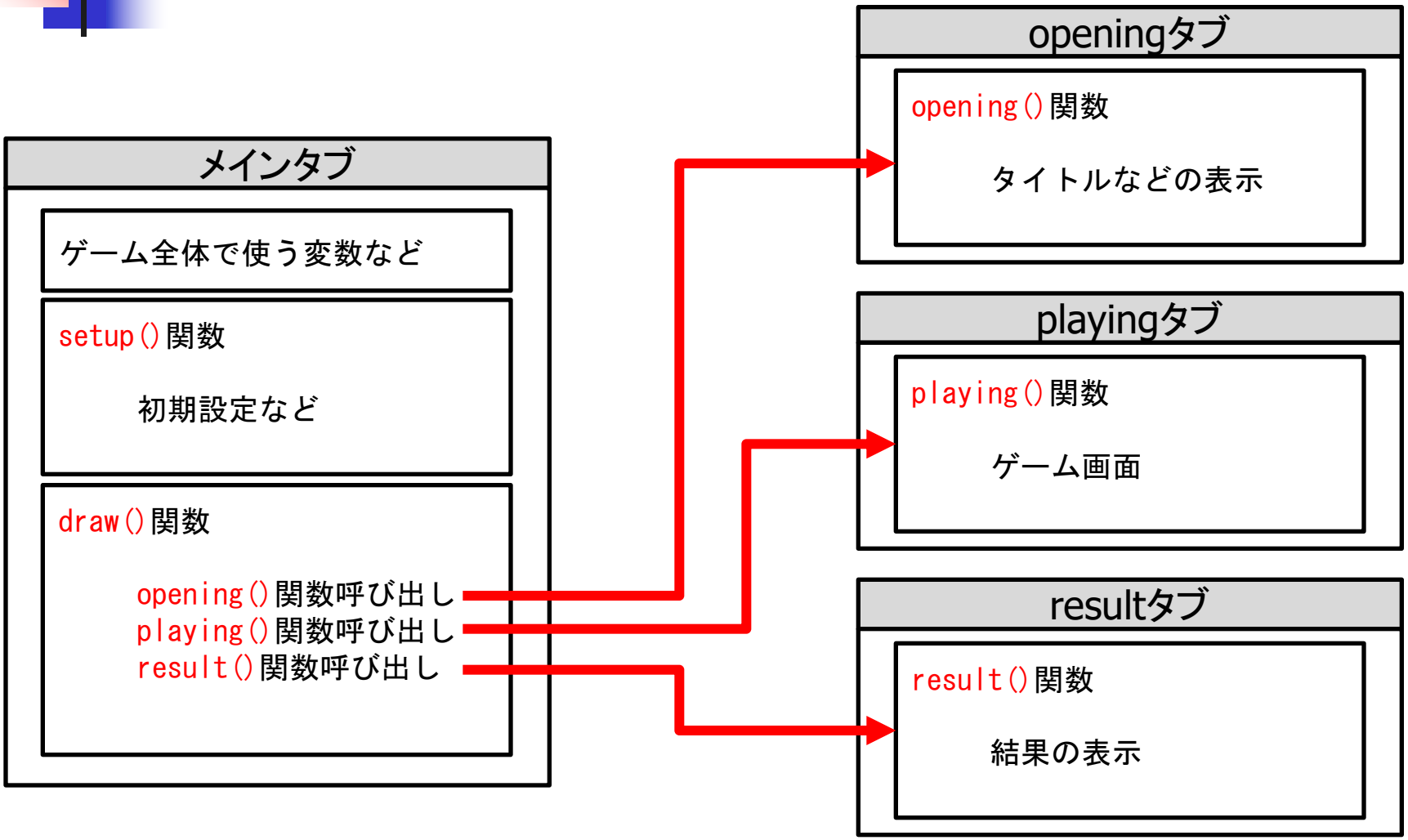
opening タブ

```
void opening() {
  if (mousePressed) {
    scene++;
  }

  background(0);

  text("whack-a-mole", 200, 200);
  text("click to start", 200, 220);
}
```

result タブ

```
void result() {
  background(0);

  if (score == 3) {
    text("You Win!", 200, 200);
  }
  else {
    text("Game Over ... ", 200, 200);
  }
}
```

プログラムの全体構造


openingタブ
opening()関数
タイトルなどの表示
メインタブ
ゲーム全体で使う変数など
setup()関数
初期設定など
draw()関数
opening()関数呼び出し
playing()関数呼び出し
result()関数呼び出し
playingタブ
playing()関数
ゲーム画面
resultタブ
result()関数
結果の表示

Step8: 改良する

- 見た目の改良
  - 色をつける
  - 画像を使う
- ゲーム性の改良
  - たたきづらくする
  - 出現位置を固定する